Docker + Portainer

必须先安装 Docker，才能装 Portainer 或其他容器服务。

因为 **Portainer 本身也是一个 Docker 容器。**
所以顺序一定是：

```
1 安装 Docker
2 启动 Docker
3 用 Docker 启动 Portainer
4 浏览器管理 Docker
```

结构其实是这样的：

```
Linux
 |
Docker Engine
 |
├─ Portainer（网页管理）
├─ MySQL
├─ Redis
├─ RabbitMQ
└─ Nacos
```

最推荐你现在这样做（运维学习路线）

你现在正在学 **Linux 运维**，所以最好的方式是：

👉 **在 Linux 上安装 Docker**，不要在 Windows 装 Docker Desktop。

因为未来公司环境基本都是：

- Ubuntu
- CentOS
- Rocky Linux

Linux 安装 Docker（非常简单）

Ubuntu，只需要几条命令。

① 更新软件源

```
sudo apt update
```

---

② 安装 Docker

```
sudo apt install docker.io -y
```

---

③ 启动 Docker

```
sudo systemctl start docker
```

---

④ 设置开机启动

```
sudo systemctl enable docker
```

---

⑤ 查看是否成功

```
docker --version
```

如果看到类似：Docker version 24.x.x

说明安装成功。

测试 Docker

运行一个测试容器：

```
sudo docker run hello-world
```

如果成功会看到：

```
Hello from Docker!
```

说明 Docker 正常。

Docker 已经安装成功了，但拉取镜像失败了

因为 `hello-world` 这个镜像需要从 **Docker Hub** 下载。

你的报错一般类似：

```
Unable to find image 'hello-world:latest' locally
docker: Error response from daemon
```

或者后面会有：

```
Get https://registry-1.docker.io ...
```

这说明 **Docker 在联网拉镜像。**

先确认 Docker 是正常的

你可以运行：

```
docker --version
```

```
sudo systemctl status docker
```

如果显示：

```
active (running)
```

说明 Docker 服务没问题。

为什么会拉不下来

在国内经常会出现：

- Docker Hub 网络慢
- 被墙
- DNS 问题

所以 Docker 需要 **配置镜像加速器。**

解决方法（强烈推荐）

编辑 Docker 配置文件：

```
sudo nano /etc/docker/daemon.json
```

如果没有这个文件就新建。

写入：

```
{
  "registry-mirrors": [
    "https://docker.m.daocloud.io",
    "https://hub-mirror.c.163.com"
  ]
}
```

保存。

然后 **重启 Docker**：

```
sudo systemctl daemon-reexec
sudo systemctl restart docker
```

重新测试

再运行：

```
sudo docker run hello-world
```

如果成功你会看到：

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

那就说明：✅ Docker 完全正常。

运维常用命令

查看 Docker 下载的镜像：

```
docker images
```

你会看到：

```
REPOSITORY   TAG       IMAGE ID
hello-world  latest    xxxxxx
```

配置Docker权限

默认情况下 Docker 需要 **root 权限**，所以你才必须写：

```
sudo docker run hello-world
```

如果想 **不用 sudo**，需要把当前用户加入 **docker 用户组**。

Docker 为什么需要 sudo

Docker 实际上是在控制：

- 容器
- 网络
- 文件系统
- 进程

这些都是 **系统级操作**，所以默认只有 **root** 可以操作。

Docker 为了解决这个问题，提供了一个用户组：`docker`

只要用户属于这个组，就可以直接运行 docker。

配置步骤（只要 1 条命令）

把当前用户加入 docker 组：

```
sudo usermod -aG docker $USER
```

解释一下：

- `usermod` → 修改用户
- `-aG` → 添加到组
- `docker` → docker 用户组
- `$USER` → 当前用户

让配置生效

执行完后需要 **重新登录**。

最简单的方法：`exit`

重新 SSH 登录，或者重新打开终端。

或者执行：`newgrp docker`

四、测试是否成功

运行：

```
docker run hello-world
```

如果成功并且 **不需要 sudo**，说明配置好了。

检查方法

查看当前用户属于哪些组：

```
groups
```

如果看到：

```
moshe docker sudo
```

说明 docker 组已经加入成功。

运维必须知道的安全点（重要）

把用户加入 docker 组其实等价于：

```
root 权限
```

因为 docker 可以：访问整个系统。

```
docker run -v /:/host
```

所以公司服务器一般只给 **管理员用户** docker 权限。

顺便告诉你一个很酷的事实

很多运维服务器其实是这样：

```
Linux
   |
Docker
   |
├── MySQL
├── Redis
├── Nginx
├── RabbitMQ
└── Nacos
```

整个服务器 **只安装 Docker**，其他服务全部用容器。

这就是现在流行的：

云原生 / DevOps 架构。

Linux安装 Portainer

一条命令：

```
docker run -d \
-p 9000:9000 \
--name portainer \
--restart=always \
-v /var/run/docker.sock:/var/run/docker.sock \
-v portainer_data:/data \
portainer/portainer-ce
```

然后浏览器打开：

```
http://服务器IP:9000
```

就能看到 Docker 管理界面。

打开浏览器

访问：http://8.153.193.212:9000

（就是你 SSH 登录的那个服务器 IP）

你会看到 **Portainer 初始化页面。**

第一次会让你：

1️⃣ 创建 **管理员密码**
2️⃣ 选择 **Local Docker**

然后就进入 Docker 管理界面了。

如果打不开网页（阿里云经常遇到）

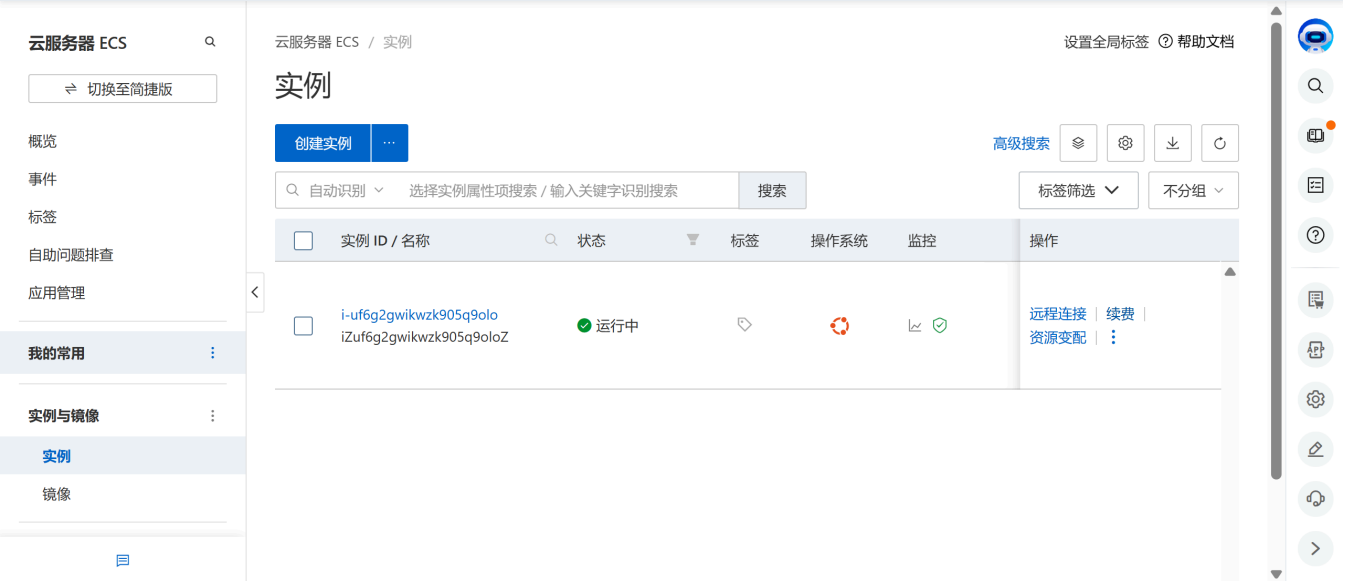
需要在 **阿里云安全组开放 9000 端口**：

路径：

```
阿里云控制台
ECS
安全组
入方向规则
```

添加：

```
端口：9000
协议：TCP
来源：0.0.0.0/0
```



点击实例

点击这一行：i-uf6g2gwikwzk905g9olo

进入 **实例详情页。**

找到安全组

进入实例后，往下找一个区域：

一般叫：网络与安全

里面会有：安全组

你会看到类似：sg-xxxxxxxx

点击这个 **安全组ID。**

进入入方向规则

进入安全组后：

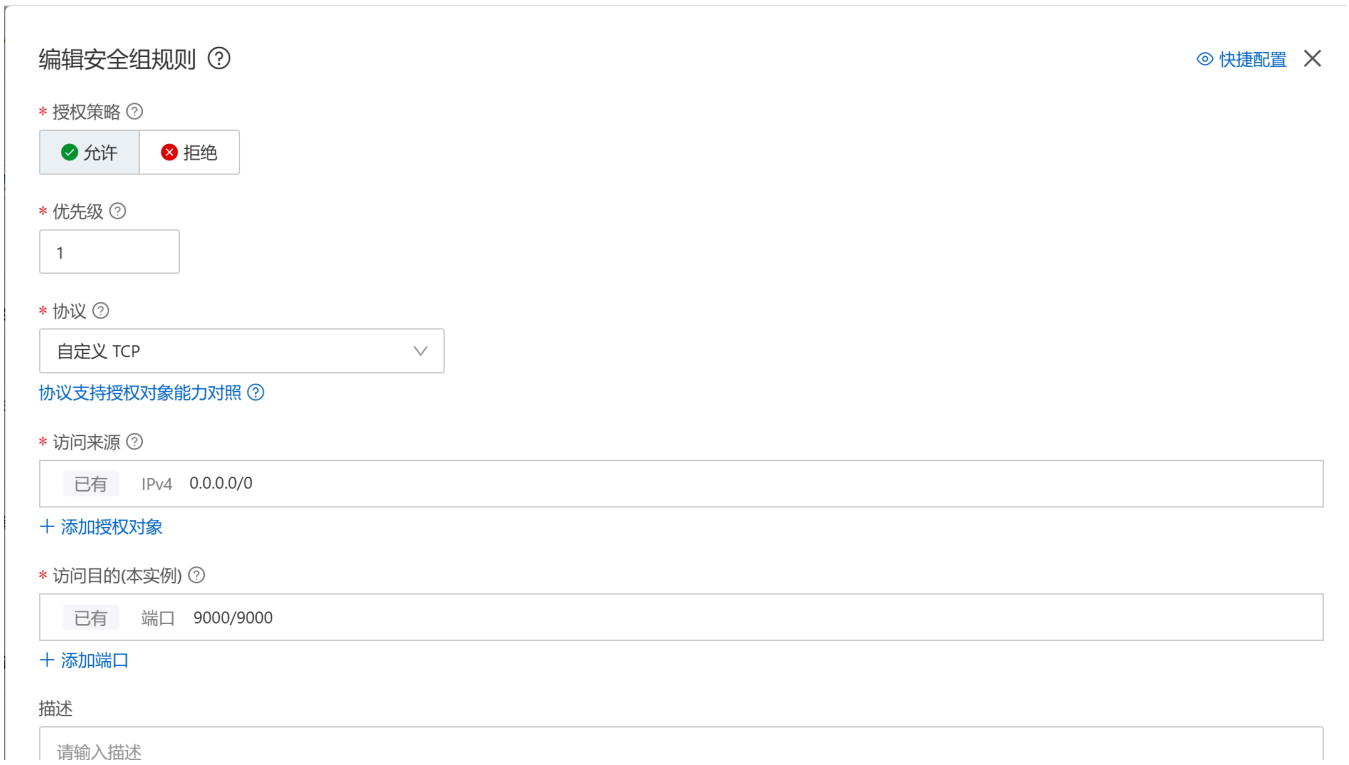
点击：入方向规则

然后点击：添加规则

填写：

项目	填写
协议类型	TCP
端口范围	9000
授权对象	0.0.0.0/0
描述	portainer

然后 **保存**。



访问来源

在这里输入：0.0.0.0/0

意思是：允许所有IP访问

因为你要从浏览器访问服务器。

访问目的（端口）

点击：新增

然后输入：9000

就是 Portainer 的端口。

最终应该是这样

项	内容
授权策略	允许
协议	TCP
访问来源	0.0.0.0/0

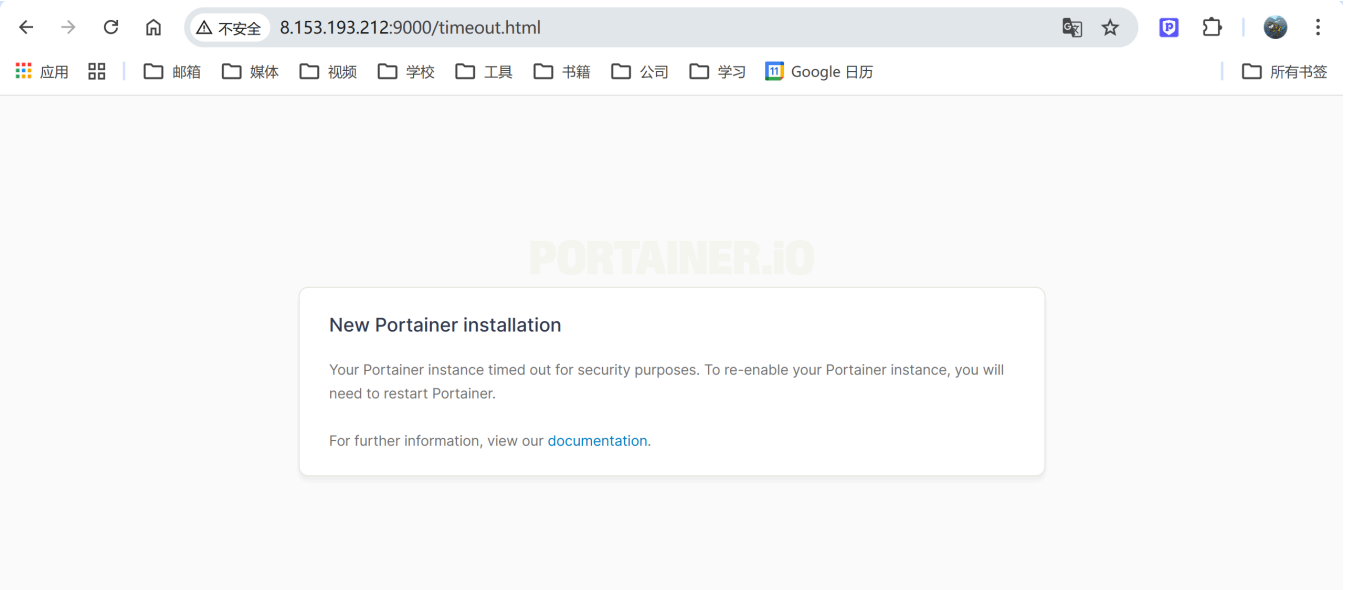
项	内容
端口	9000

然后点击：确定

访问 Portainer

规则保存后，打开浏览器：

```
http://8.153.193.212:9000
```



已经成功访问到 Portainer 了，只是它提示需要重启容器。

你看到的提示：

> **Your Portainer instance timed out for security purposes**

意思是：

- Portainer 初始化页面 **长时间没有操作**
- 为了安全 **自动锁定**
- 需要 **重启 Portainer 容器**

这不是错误，只是 **安全机制。**

解决方法

回到你的 **服务器终端**，输入：

```
docker restart portainer
```

如果成功会看到：

```
portainer
```

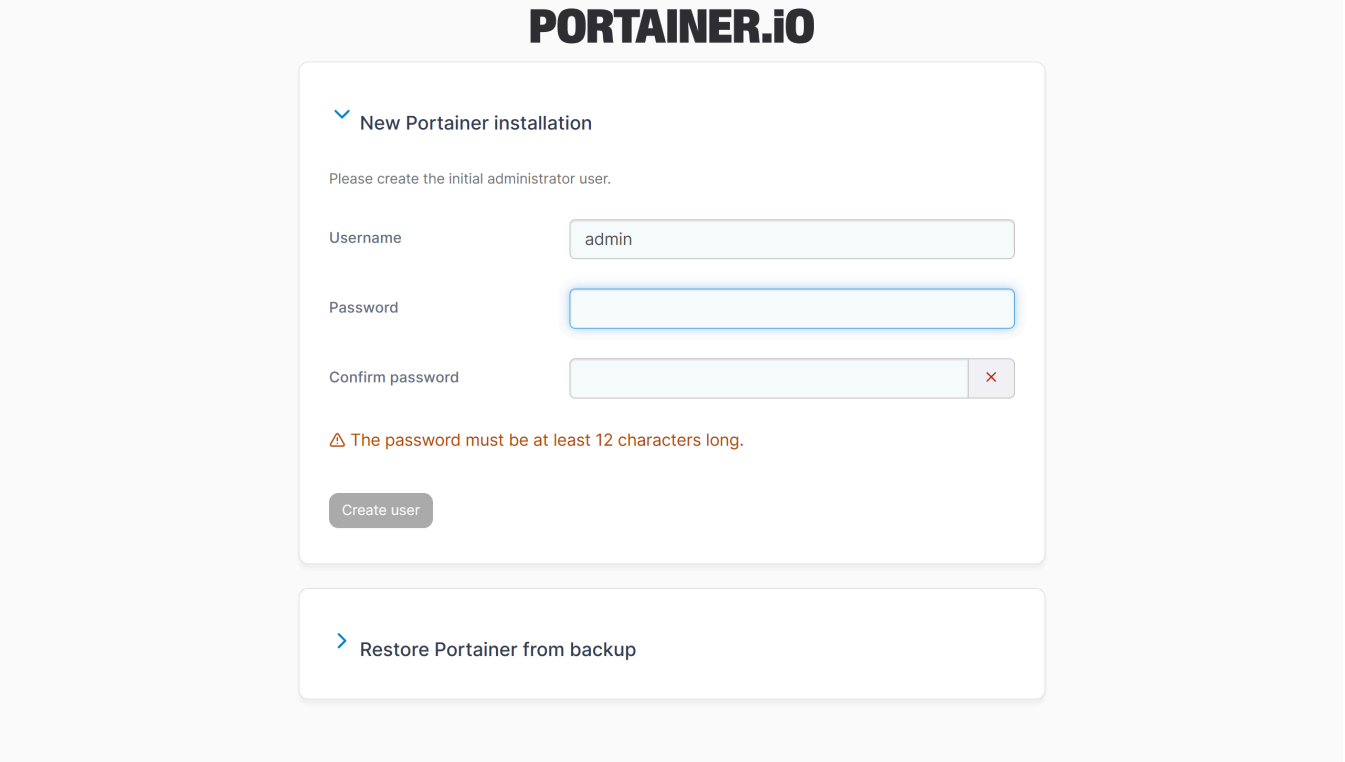
然后重新访问

刷新浏览器：

```
http://8.153.193.212:9000
```

你就会看到真正的初始化页面：

```
Create administrator user
```



Portainer网页端配置

1 创建管理员密码

输入你自己的密码。

Qwqzad2007--

2 选择环境

选择：Local

然后进入管理界面。

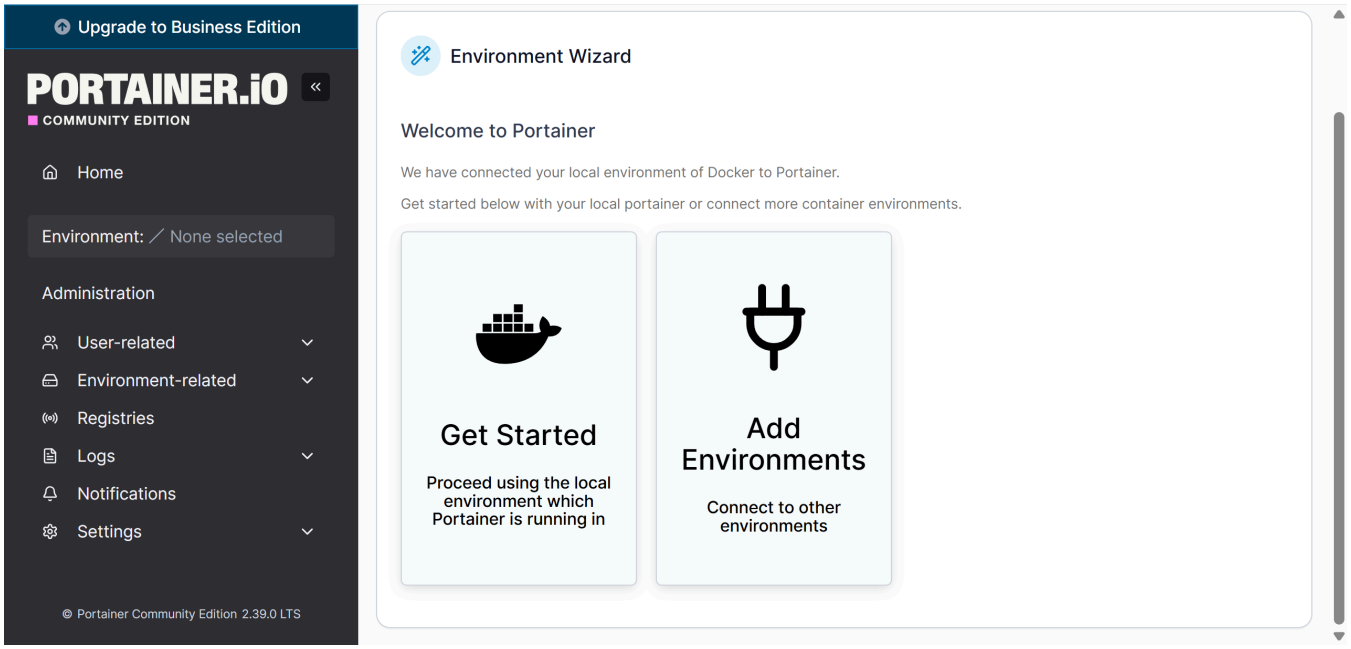
成功后你会看到

Portainer 的主界面：

```
Containers
Images
Volumes
Networks
Stacks
```

以后你可以 **网页管理 Docker**，比如：

- 启动容器
- 查看日志
- 创建 MySQL
- 创建 Redis
- 部署服务



现在看到的是 **Environment Wizard（环境向导）** 页面。

其实只需要点一个按钮就行。

下一步操作

点击左边这个：**Get Started**

意思是：

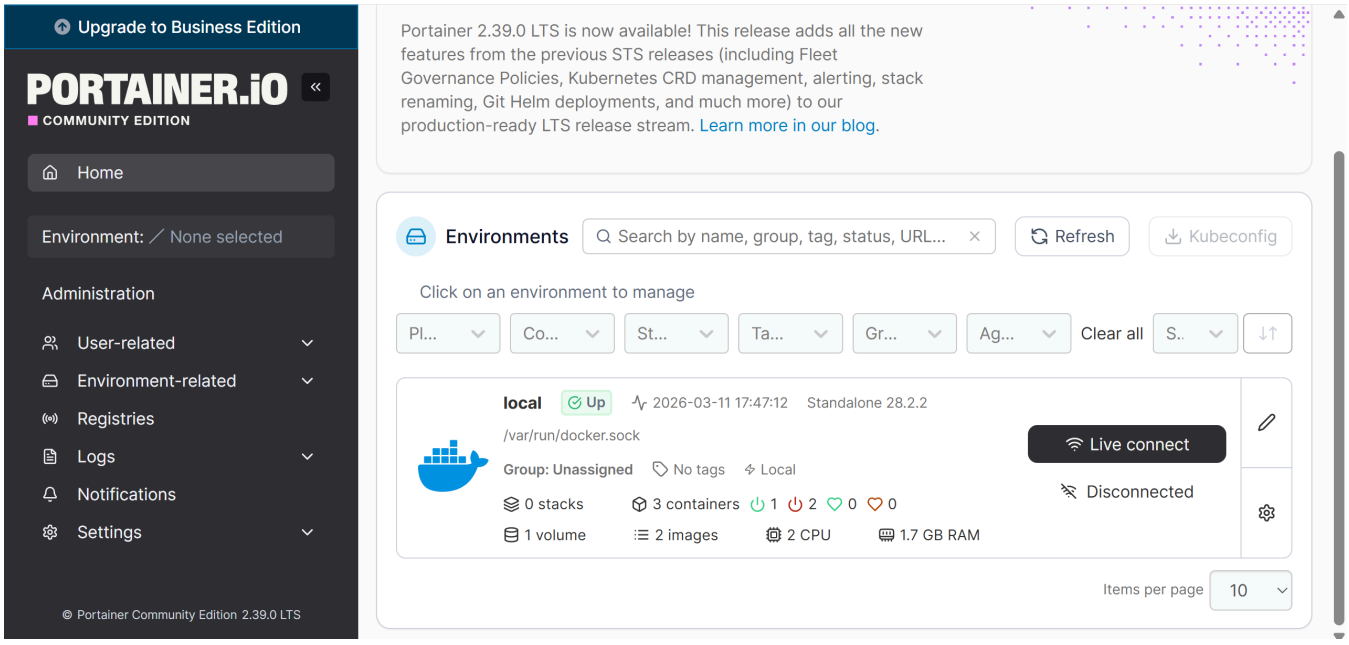
> 使用当前服务器的 Docker 环境。

因为 Portainer 已经检测到：

```
This Portainer instance is running in
local Docker environment
```

也就是说：

```
Portainer
    |
Docker
    |
你的服务器
```



这里显示的是 **Docker 环境列表。**

你现在有一个环境：local

意思是：

```
Portainer
    |
Docker Engine
    |
你的服务器
```

状态：Up

说明：

- Docker 正在运行
- Portainer 成功连接 Docker

下面显示：

```
3 containers
2 images
1 volume
```

意思是：

- **3个容器**
- **2个镜像**
- **1个数据卷**

这很正常。

下一步应该点什么

点击右边这个按钮：Live connect

进入 Docker 管理界面。

进去之后你会看到

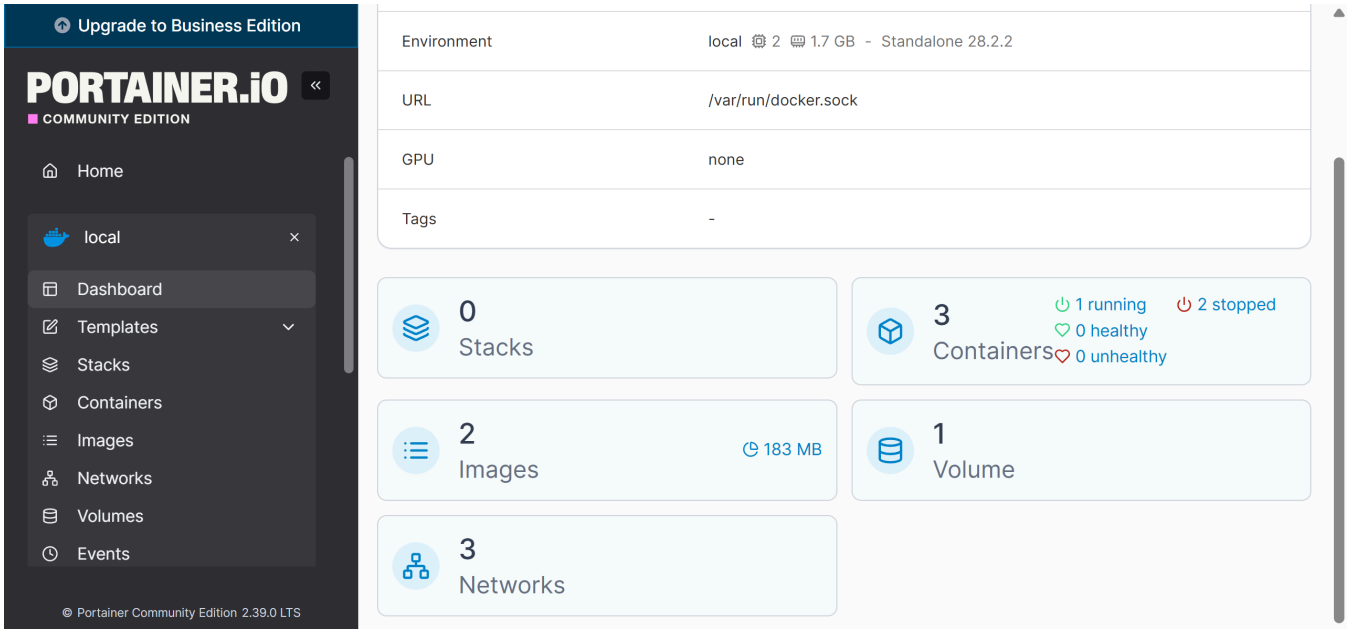
Docker 的核心管理界面：

左边会出现：

```
Containers
Images
Volumes
Networks
Stacks
```

解释一下：

功能	作用
Containers	容器
Images	镜像
Volumes	数据卷
Networks	网络
Stacks	docker-compose



现在服务器的 Docker 状态

从页面可以看到：

```
3 Containers
1 running
2 stopped
```

意思是：

- **3 个容器**
- **1 个正在运行**
- **2 个停止**

通常情况是：

容器	状态
portainer	running
hello-world	stopped
可能还有一个测试容器	stopped

因为 `hello-world` 执行完就会退出。

Images（镜像）

```
2 Images
```

说明你现在服务器已经下载了两个镜像，例如：

```
portainer/portainer-ce
hello-world
```

Docker 的逻辑是：

```
镜像 = 程序模板
容器 = 运行中的程序
```

就像：

```
Java jar → 运行后变成进程
Docker image → 运行后变成 container
```

Volume（数据卷）

```
1 Volume
```

这个是给 Portainer 存数据的。

我们刚刚创建时用了：

```
-v portainer_data:/data
```

所以会看到一个 Volume。

Network（网络）

```
3 Networks
```

Docker 默认有三个：

```
bridge
host
none
```

用于容器之间通信。

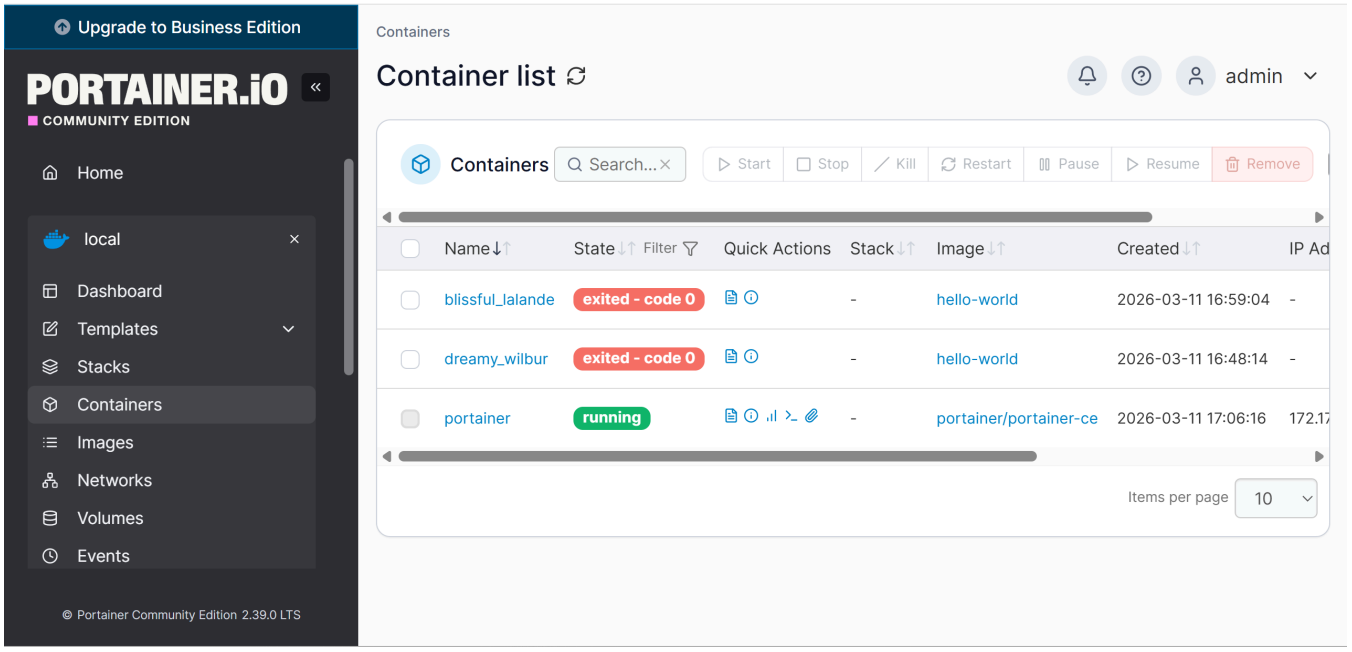
部署一个 Redis 容器

```
缓存数据库
SpringBoot / Java / 微服务必用
```

用 Portainer 部署 Redis（超简单）

点击左边：Containers

然后点击：Add container



填写：

Name

```
redis
```

Image

```
redis
```

然后找到 **Publish a new network port**

填写：6379

然后点击：Deploy the container

成功后服务器会变成

```
Docker
 ├ portainer
 ├ redis
```

你的 Redis 地址就是：

```
8.153.193.212:6379
```

你可以直接连接 Redis

比如：

Linux

```
redis-cli -h 8.153.193.212
```

Java / SpringBoot

```
host: 8.153.193.212
port: 6379
```

未来你会这样用

以后你部署服务就会变成：

MySQL

```
docker run mysql
```

Redis

```
docker run redis
```

Nginx

```
docker run nginx
```

RabbitMQ

```
docker run rabbitmq
```

全部隔离运行。

我建议你做一件事（非常适合运维）

你现在可以搭一个 **运维实验环境**：

```
Linux
 |
Docker
 |
Portainer
 |
├─ MySQL
├─ Redis
├─ RabbitMQ
├─ Nacos
└─ Nginx
```

这基本就是 **微服务环境的基础架构。**